

장영진 Youngjin Jang (Vincent)

Backend Engineer · Web2 / Web3 · 5+ yrs

PHONE +82 010-7472-0453 EMAIL vincentjangdev@gmail.com SITE vincentjang.dev GITHUB JangVincent
BLOG dev.to/vincentjang LINKEDIN vincentjang

SUMMARY

스타트업 현장에서 기획부터 운영까지 폭넓게 경험한 **백엔드 개발자**입니다. NestJS·Bun·Go·Spring 등 다양한 런타임을 다루며 **대규모 트래픽 처리, 기술 부채 해소, 주도적 아키텍처 설계** 경험을 쌓아왔습니다. Web2 SaaS와 Web3 온체인 시스템을 모두 구축해 본 풀스택 백엔드 엔지니어이며, **Zero-to-One 단계 스타트업의 코어 시스템을 오너십을 갖고 이끌어 본** 경험에 강점이 있습니다.

"스스로만 빛나는 개발자보다, 달처럼 다른 이들의 빛과 함께 화합하는 개발자."

EXPERIENCE

AIM-Intelligence — Backend Manager

2025.11 — 2026.06

기업용 AI 보안-거버넌스 SaaS. **Starfort**(컨트롤 플레인)와 **Bastion-Guardian**(런타임 가드)의 **v1.2까지 직접 지휘하며 설계·구축**. 6개월 안에 **출시 → Series A 라운드 클로징 건인 → 엔터프라이즈 도입(SaaS · On-Premise 양쪽 지원)** 까지의 사이클을 완수. 두 리포 합산 974 커밋 중 약 95% 본인 작성. aim-intelligence.com

주요 파트너 · LG · 하나 · 교보 등 국내 통신·금융 엔터프라이즈

Starfort — Admin Control Plane

- **AWS IAM에서 영감받은 자체 Policy Engine 주도 설계** — Role-only는 fine-grained 인가가 불가, IAM-only는 계층 표현이 어색하다는 한계를 인식. **Role scope(Company → Organization → Project) 자동 계층 게이트와 IAM 스타일 정책 statement를 결합한 2-stage · deny-first 평가 모델**로 표현력과 운영 단순성 동시 확보. **자체 SRN(Starfort Resource Name) 리소스 식별 포맷**·와일드카드·변수 확장·정규식 이스케이프 기반 ReDoS 방어·condition operator(StringEquals, DateGreaterThan 등)까지 구현, 단위 테스트 40KB(1,000+ 라인) 분량의 회귀 안전망 구축.
- **외부 AI 모니터링 시스템(Langfuse 계열)과의 멀티 테넌트 dual-write 일관성 해결** — Redis·SQS 등 외부 큐 추가 없이 PostgreSQL 위에 **Transactional Outbox 패턴**을 직접 구현. 도메인 mutation과 동일 트랜잭션에서 메시지 적재, **FOR UPDATE SKIP LOCKED** 기반 큐 처리로 멀티 인스턴스 안전성과 at-least-once를 동시 달성. 지수 백오프 재시도·visibility timeout 기반 크래시 워커 락 회수·역등성 키까지 포함.
- **3-tier 인메모리 캐시 + Single-flight 패턴으로 hot path DB 부하 최소화** — Bastion-Guardian 런타임이 정책·엔진·캡처를 매 요청마다 fetch해야 하는 read-heavy 특성에 대응. 동시 cache miss는 in-flight Promise 공유로 DB 호출 1회로 합치고, 정책 변경 시 TTL 만료를 기다리지 않는 **이벤트 기반 push 무효화**를 적용. 무효화 실패에는 지수 백오프 재시도와 구조화 로그 태그를 갖춰 stale 위험을 운영 단계에서 즉시 추적 가능하도록 설계.
- **읽기/쓰기 도메인 분리 모노레포 구조 설계** — admin(쓰기·관리)과 bridge(런타임 read-side)를 별개 앱으로 분리해 캐시 정책·배포 사이클을 독립 운영. EKS Helm(OCI 차트) + GitHub Actions OIDC 무자격증명 CI/CD, 라벨 기반 PR별 프리뷰 이미지 빌드 파이프라인 구성.

□ **규모** · 47 Prisma 모델 · 35 컨트롤러 · 44 서비스 · 이중 DB(Starfort · Langfuse 계열) · 본인 주도 95%

□ **Stack** · NestJS · Prisma · PostgreSQL · AWS EKS · Helm · External Secrets · GitHub Actions

| Bastion-Guardian — GenAI DLP Gateway

- **hot path 응답 20ms 이하 달성 (k6 스트레스 테스트 실측)** — 사내 직원이 ChatGPT·Copilot 응답을 기다릴 때 가드가 체감 지연을 만들지 않아야 한다는 제품 요구를 인식. **에이전트 상태 early-exit → URL blacklist/whitelist 선분기 → 본 검사(AI 분석 엔진) 호출** 순의 short-circuit 파이프라인과 인메모리 캐시로 요청당 설정 fetch 왕복을 제거. 미등록 URL은 차단하는 **default-deny**를 판정 불변식으로 유지.
- **Bun 런타임 + 단일 바이너리(bun build --compile) + distroless 이미지 채택** — 콜드 스타트 단축과 공격 표면 최소화를 동시에 노린 결정. HTTP 프레임워크 없이 **Bun.serve 네이티브 라우팅**과 런타임 내장 multipart·해시·gzip 기능을 활용해 런타임 의존성을 5개로 유지, 셸 없는 distroless 베이스에서 동작.
- **비균질 AI 프로토콜의 user input 추출을 정책 데이터만으로 통일하는 파싱 파이프라인 설계** — ChatGPT(JSON)·M365 Copilot(SignalR WebSocket)·Claude·Gemini-multipart-base64 등을 **14종 파싱 step method 조합**으로 처리. **순수 JS jq 미니 인터프리터(렉서·파서·평가기·AST 캐시)**를 처음부터 구현해 프로세스 spawn·WASM·외부 jq 의존성 0. 새 AI 서비스 추가 시 코드 변경 없이 파싱 정책 추가만으로 대응.
- **회사·프로젝트·에이전트 단위로 정확하게 무효화되는 3-Tier 캐시 구조** — Global(30분) / Tenant(5분) / Identity(5분) 계층별 TTL과, **agentId(UUID)·API key 해시 → 회사 ID 역인덱스**로 요청 시점에 회사 ID를 모르더라도 정확한 scope 단위 invalidate. **generation counter 기반 race-safe 쓰기**로 fetch 도중 무효화가 도착해도 stale 데이터가 캐시에 남지 않도록 설계.
- **Desktop / API 이중 게이트웨이 설계** — Desktop Guard는 **multipart 양방향**(요청 form-data → 응답 multipart/mixed: 검사 결과 JSON + sanitized 파일), API Guard는 **OpenAI Chat Completions 호환 시그니처**를 미러링해 SDK 사용자가 코드 수정 없이 API Key 헤더만으로 동일 PII·기밀 검사를 적용.
- **마스킹된 요청을 원문 그대로 복원하는 turn 단위 mask/unmask 라이프사이클 구현** — INPUT 패킷에서 마스킹한 토큰 매핑을 turn_id로 저장했다가 모델 OUTPUT 패킷에서 복원. **longest-token-first 치환**으로 마스킹 토큰 간 오매칭을 방지하고, 만료·재시작 시에는 silent graceful-degrade로 사용자 흐름을 끊지 않도록 설계.
- **JWT 기반 Desktop Agent 인증 흐름 직접 설계** — agentId(UUID v4) + Company Access Key로 등록하고, 미할당 (UNASSIGNED) 에이전트에도 **minimal claim JWT**를 발급해 silent recovery로 자연스럽게 재등록. 서명 키 로테이션 시 전용 에러 코드로 에이전트 self-cleanup을 유도해 운영 개입 없이 토큰 무효화를 전파.

□ **Stack** · Bun · zod · jose · pino · Langfuse · Helm · distroless · GitHub Actions

Gomble — Tech Lead

2024.12 — 2025.10

Web3 게임/NFT 커뮤니티 플랫폼. **개발 리드**로서 **DAU 1만+** 환경에서 대규모 이벤트 트래픽(TGE, Launchpool 등)을 안정적으로 처리하며 서버 인프라와 코드 품질을 개선.

| Gomble Squad / Mini Game Server / TGE Server / CashPop / Mew

- **TGE 이벤트에서 4~60K req/sec 대규모 트래픽 처리** — 기존 NestJS + Express 조합으로는 단일 코어 처리량의 한계가 명확했음. NestJS의 **platform-agnostic** 특성을 살려 **HTTP 엔진을 Fastify로 교체**해 단일 코어 처리량을 약 15% 향상하고 pm2 cluster 모드로 멀티 코어 확장. GET 비중이 높은 워크로드 특성에 맞춰 **PostgreSQL read replica(master/slave) 구성**과 **Redis cache-aside**로 DB 부하를 분산.
- **이벤트 트래픽에서의 동시성·운영 안전장치 직접 설계** — 지갑 로그인·미션 클리어 등 경합 구간에 **Redlock 분산락 + 자체 구현 in-process Key-lock**의 2중 동시성 제어를 적용해 중복 유저 생성·중복 보상 지급을 차단. Redis 플러그를 폴링하는 **circuit breaker 글로벌 가드**로 재배포 없이 즉시 전체 API를 점검 모드로 전환할 수 있게 하고, 5xx 발생 시 Slack 실시간 알림으로 이벤트 중 장애 대응 시간을 단축.

- **반복되던 신규 서비스 세팅 비용을 팀 단위로 제거 — 사내 5개 프로덕트가 사용한 공용 서버 템플릿 주도 정립** — 새 프로덕트마다 밑바닥부터 세팅을 반복하는 비효율을 인식. 코드 컨벤션(lint-import 정렬·모듈 스캐폴딩 스크립트)과 DB 레이어 표준(TypeORM CLI용 dataSource export·마이그레이션 스크립트, 이후 Prisma 버전은 연결 라이프사이클을 중앙화한 공용 DatabaseService)을 템플릿에 내장. 핵심 로직에는 테스트 코드를 동봉해(IAP 결제 검증 스펙 2,100+ 라인 등) 신규 프로덕트의 기본 품질선을 확보.
 - **Game-Fi 플랫폼 트랜잭션 안정화** — 산발적으로 발생하던 트랜잭션 일관성 깨짐 이슈를 **transactional EntityManager 기반 리포지토리 접근으로 일원화**해 해결. 정원이 있는 스쿼드 참여·보상 수령 등 제한 자원의 동시 접근에는 **자체 구현 Key-lock(키 단위 in-process 뮤텝스)**을 도입, 자원별 단일 키 직렬화로 경합과 데드락 여지를 제거.
 - **인게임 재화의 가치가 암호화폐와 직결되는 환경에서의 데이터 무결성 baseline 확보** — 단순 저장소 역할이던 미니게임 통합 서버에 **유저별 키 기반 HMAC-SHA256 요청 무결성 검증**(변조된 게임 클라이언트 요청 차단)과 **단일 활성 세션 강제**(중복 로그인 토큰 무효화), 서버 간 ingress 서명 검증을 도입해 보안 baseline을 새로 설계.
 - **실물 가치가 오가는 결제·정산 플로우의 서버사이드 검증 구현** — Worldcoin World App 기반 프로덕트에서 **World ID(Orb) 인증**과 포인트→토큰 redeem 시 **서버 서명(keccak256 + 1회용 nonce) 발급 + 온체인 영수증의 ERC-20 Transfer 이벤트·금액 교차 검증**을 구현. 모바일 게임 서버에는 Apple/Google **IAP 영수증 서버사이드 검증** 파이프라인을 구축.
 - **Redash 도입으로 이벤트 운영 의사결정 리드타임 단축** — 대규모 이벤트 때마다 개발팀이 일회성 SQL을 작성해주던 비효율과, PM·BD가 실시간 지표를 보지 못해 의사결정이 지연되던 문제를 동시에 인식. **Redash 인프라를 직접 셋업하고 프로덕트별 핵심 지표 대시보드를 정의**해, 비개발 직군이 직접 쿼리·대시보드를 운영하면서 개발팀은 코어 작업에 집중할 수 있도록 분리.
- **Stack** · NestJS · TypeORM / Prisma · PostgreSQL · Redis (ioredis · Redlock) · pm2 · ethers · viem · AWS ECR/ECS · GitHub Actions

Catze Labs — Backend Developer

2022.08 — 2024.10

Web3 게임 퍼블리싱 플랫폼. 입사 초기부터 플랫폼·솔루션 아키텍처를 담당하며 해외 협업사들과 기획부터 긴급 대응까지 전방위 수행. Web3 해커톤 4회 입상, **회사의 Consensys 투자 유치에 실질적 기여**.

Web3 Game Publishing Platform

- **입금 처리 시간 15분 → 15초, 60배 단축** — 기존 cron 기반 입출금 스크립트는 온체인/오프체인 시간차 15분, 스캠 데이터 미대응, RPC rate limit 등 누적된 부채를 갖고 있었음. 주기 폴링 방식을 그대로 두고는 UX 개선이 불가능하다고 판단, **Bull 메시지 큐 기반 비동기 처리**로 입금 시스템을 재설계 — 클라이언트가 제출한 트랜잭션 영수증과 독립 RPC 인덱서가 적재한 온체인 로그를 **교차 검증**해 입금을 확정하고, 체인별 confirmation delay를 DB 설정으로 제어. 스캠 트랜잭션은 토큰 컨트랙트 주소의 log event 검증으로 필터링하고, 실패 건은 상태 머신 + 수동 재처리 이력으로 추적하며 **출금 실패 시 인게임 재화 자동 환불 롤백**까지 포함한 fail-safe 구조 설계.
- **멀티체인 온체인 인프라 직접 구현** — ETH·Linea·BSC·Manta·Oasys 등 체인별 인덱서(Covalent·Blockscout)를 커서 **재개형 듀얼 소스 인덱서**로 통합하고, viem 기반 **배치 출금(단일 tx 다중 수취)·버전 관리형 NFT 홀더 스냅샷 인덱싱·nonce + EIP-712 지갑 서명 인증**을 구현. **BRC-20(Bitcoin Ordinals)** 민팅·브릿징으로 EVM 밖 생태계까지 지원 확장.
- **사내 통합 백오피스 주도 구축** — 비개발 직군이 직접 운영, 개발 리소스 절감 — 해외 협업사와의 멀티체인 이벤트에 운영 도구가 없어 개발자가 운영 업무까지 떠안던 문제를 인식. 프론트엔드와 협업해 6개 프로덕트를 한곳에서 관리하는 백오피스(약 120개 엔드포인트)를 직접 설계·구축. **Google OAuth + Slack DM OTP + QR 기반 TOTP 2FA + 프로덕트 단위 role 접근 제어**로 보안을 다층화하고, 온체인 입금 큐 컨슈머·GitHub PR 예약 자동 머지·버전 관리형 이메일 템플릿(Mailgun 큐) 등 운영 자동화를 내장. 결과적으로 비개발 직군이 운영을 직접 처리할 수 있게 되어 멀티 프로덕트 가시성과 개발 리소스 모두 확보.

- **사내 공용 라이브러리 2종(Logger / Crypto) 주도 개발** — **GitHub Packages** 배포, **사내 5개+ 서비스 채택** — 서비스 로그 모니터링 부재와 임계값 우회 요청 등 누적된 관측성·보안 부채를 해소. **Logger**는 전체 라우트의 요청을 UUID로 상관관계 추적하고 에러 컨텍스트(stack·함수명·argument) 기록을 데코레이터로 표준화, Ops와 협업해 Loki + Prometheus + Grafana 대시보드를 구축. **Crypto**는 전자봉투(RSA-2048 + AES-256-CBC + SHA256 서명) 방식을 endpoint 단위 데코레이터로 제공하고, Redis 기반 유저별 공개키 레지스트리까지 포함.
- **Web3 해커톤 4회 입상 및 그 결과를 발판으로 Consensys 투자 유치 기여** (상세는 Awards 섹션).
- **Redash 도입** — 비개발 직군이 BD/마케팅 자료로 직접 활용할 수 있도록 프로젝트별 대시보드를 Ops 엔지니어와 함께 구축.
- **Stack** · NestJS · Prisma · Bull · MySQL · Firestore · Redis · viem · GCP Cloud Run · Kubernetes (GKE) · ArgoCD · GitHub Actions (Workload Identity)

VSQUARE — Software Engineer

2020.12 — 2021.12

CMS·LMS 솔루션 풀스택 개발. 7개 프로젝트 동시 진행 환경에서 자체 학습과 문서화 문화 도입으로 개발 효율을 끌어올림, 회사가 예년 수익을 2분기 만에 달성하는 동안의 개발 역량을 맡음.

- **자생한방병원 / 어린이 급식관리지원센터 / 한국여성인권진흥원 LMS / 정화예대 홈페이지·학사정보 / 시그너스(E-commerce)** 등 7개 프로젝트 BE/FE 풀스택 담당.
- **대표 작업** — 단체 이메일 일괄 발송, 수수료 PDF·Excel export(Apache POI), 식단표 드래그앤드롭 편집기, **이니시스 + PayPal** 다중 PG 연동으로 글로벌 결제까지 포함한 결제 시스템 구축 등.
- **Stack** · Java · Spring Boot · MyBatis · jQuery · Vanilla JS · Jenkins

FREELANCE

Waifu Sweeper — Backend Lead (외주, YGG 협업)

waifusweeper.fun

Web3 지갑 기반 Minesweeper RPG. **Abstract zk-rollup** 체인에 NFT/SBT/Shop 컨트랙트, **YGG (Yield Guild Games)** 글로벌 길드와 Partner API 협업. 백엔드 60/61 커밋 단독. **DAU 1만+ / 누적 결제액 1억 원+**.

- **스마트 컨트랙트 지갑(AA Wallet) 대응 EIP-1271 기반 인증 풀스택 설계** — 단순 ECDSA recover로는 Abstract Global Wallet 같은 컨트랙트 지갑을 지원할 수 없는 한계를 인식, **표준 EIP-1271 매직 밸류 온체인 검증** 기반 인증 흐름을 설계. 로그인 nonce는 삭제 카운트 검증으로 멀티 인스턴스 replay를 차단하고, 동시 로그인과 zombie account 문제는 **argon2id 토큰 해시 저장**으로 해결. JWT verify 단계에서는 algorithm confusion 공격을 차단하는 단일 알고리즘 강제 적용.
- **온체인 결제 검증 4중 게이트 — txHash 정규화로 중복 크레딧 0건 유지** — 영수증 로그를 발신·수신 주소·가격·패키지 ID 4개 축으로 교차 검증, 트랜잭션 해시 표기 차이(체크섬 vs 소문자)에서 발생하는 중복 크레딧 가능성을 소문자 정규화 + DB unique 제약으로 원천 차단.
- **EIP-712 typed data 결제 서명자 구현** — 서버가 발급한 typed data 서명과 **1회용 nonce**를 결합해 결제 위변조를 차단, 클라이언트가 임의로 가격을 조작할 수 없는 흐름 설계.
- **멀티 인스턴스 환경에서의 race condition을 막는 Postgres 기반 분산락 시스템** — 외부 락 인프라 없이 Postgres unique constraint와 TTL 기반 자동 해제로 분산락을 직접 구현. 게임 액션·포인트 차감·크론 leader election 등 동시성 위험 지점에 적용. revert 플로우는 검증과 업데이트를 단일 트랜잭션 안에 묶어 TOCTOU 취약점을 방지.
- **YGG Partner API + 38개 Quest 시스템 — HMAC-SHA256 양방향 서명과 30초 dedup 분산락으로 파트너 통신 보안 확보** — 서명 비교는 timing attack에 대비한 **constant-time 비교**로 구현. EventEmitter 기반 비동기 디커플링으로 quest clear 콜백을 처리하고, 포인트 차감/리펀드는 race-safe revert 트랜잭션과 **balanceBefore/After**를 기록하는 **포인트 원장**으로 정확성과 감사 가능성을 보장.

- **AWS Terraform IaC 직접 구축 + GitHub Actions OIDC 무자격증명 CI/CD** — 멀티 AZ VPC, RDS, ALB, ECS, S3, CloudFront, ACM까지 전체 인프라를 코드로 관리.

□ **Stack** · Bun · Elysia · Prisma · viem · ethers · Abstract chain · AWS · Terraform

PERSONAL PROJECTS

현장에서 마주친 **페인 포인트를 빠르게 PoC하고 프로덕트화**해 사내 생산성을 끌어올리는 패턴을 좋아합니다. **AI 등 신기술과 실무 현장 사이의 간극을 줄이는 것**이 모든 프로젝트의 공통 동기 — 모두 단독 개발했고, **회사 동료들이 실제 업무에 사용 중**입니다.

Coagent — Multi-Agent Chat (CLI + Desktop App)

cli · app

Claude Code가 1:1 대화에 갇혀 있어 여러 프로젝트를 오가며 일하기 어려웠음. 사람과 여러 에이전트가 **같은 채팅방에서 멘션으로 협업**하는 환경이 필요해 직접 만들어 사내에서 함께 사용 중.

- **CLI (OSS 코어)** — Claude Agent SDK가 plain-text 응답을 자동으로 채팅에 전달하지 못하는 문제를 인식, **단일 도구만 노출하는 MCP 서버를 직접 구현해 메시지 출구를 강제**. 에이전트가 도구를 호출하지 않고 끝낸 경우에도 결과 텍스트를 폴백으로 흘려보내 "응답이 사라지는 이슈"를 차단. 멀티 에이전트 동시성에는 single-flight 큐와 슬래시 컨트롤(컨텍스트 정리, 일시정지/재개, 강제 종료 등 9종)로 운영성 확보.
- **Desktop App (엔드유저 제품)** — 패키징된 GUI에서는 SDK가 system node를 spawn하지 못하는 문제를 인식, **Electron 바이너리를 Node 런타임으로 재사용하는 패턴(RunAsNode 플러그인 + asar unpack + macOS Finder 쉘 PATH 복원)**으로 안정화. **Claude·Codex(OpenAI)**를 단일 **AgentBackend 인터페이스로 추상화한 듀얼 백엔드**와 in-process SDK MCP + bearer 토큰 인증 HTTP Streamable MCP 2종 트랜스포트를 운영. CLI 코어와 코드 분리해 엔드유저 GUI 레이어를 별도 운영, 자동 업데이트와 세션 resume까지 포함.

□ **Stack** · CLI: Node + TypeScript + WebSocket + Ink · App: Electron + Svelte 5 (runes) · 공통: Claude Agent SDK · MCP

Moonshine — Local Knowledge Graph Desktop App

github

결정·문제·인사이트가 노트 도구마다 흩어져 있어 연결이 보이지 않음. **로컬 SQLite + AI 자동 종류 파이프라인**으로 캡처만 하면 임베딩·관계 추출·그래프 시각화가 백그라운드에서 자동 처리되도록 직접 설계. 서버 0, 사용자 BYO API key.

- **Tauri + Svelte + Rust로 macOS / Windows / Linux 단일 코드베이스 단독 개발** — Homebrew Cask 채널을 포함한 멀티플랫폼 자동 배포 파이프라인까지 직접 구성.
- **sync DB lock과 async AI 호출을 데드락 없이 인터리빙하는 파이프라인 설계** — SQLite는 블로킹, AI 호출은 비동기인 비대칭 환경에서, Mutex 가드가 await 경계를 넘지 못하는(!Send) 제약을 **스코프 분리 패턴(잠금 → 소유 데이터로 복사 → 해제 → await → 재잠금)**으로 풀어 단일 SQLite 커넥션으로도 batch 처리와 진행률 추적을 양립.
- **OpenAI / Gemini 멀티 LLM 프로바이더 추상화** — 임베딩 차원(1536 강제 정렬)·배치 크기(2048 vs 100)·task type 차이를 단일 인터페이스 뒤로 흡수, 런타임 전환과 재임베딩/재추출 트리거까지 통합.
- **벡터 DB 없이 임베딩 검색을 직접 구현** — f32 BLOB 직렬화 + 코사인 유사도 top-K 배치 매칭과 쿼리 임베딩 캐시(동일 검색어 재사용 시 API 호출 생략)로 외부 인프라 0으로 시맨틱 검색 구성. 키워드 검색은 CJK 대응 trigram 토크나이저 기반 FTS5로 보완.
- **Wayland 글로벌 단축키 우회로 OS-specific UX 동등화** — 컴포지터 단축키가 앱 레벨에서 잡히지 않는 환경을 위해 **DBus 인터페이스를 직접 노출**, 사용자가 컴포지터 키바인딩으로 호출하도록 가이드.

□ **Stack** · Tauri · Svelte (runes) · Rust · SQLite (FTS5) · OpenAI · Gemini · Homebrew 배포

Luhmann의 카드박스 방법론을 따르고 싶었지만 기존 도구는 폴더 분류 중심이라 **노트 간 연결성**이 잘 보이지 않음. 터미널에서 글쓰기 → 링크 → 그래프 탐색까지 한 번에 끝나는 도구가 필요해 직접 구현.

- **Bun + TypeScript, 156 테스트 · 264 assertion, 12 릴리스**(v0.1.0 → v0.2.9).
- **3-tier 노트 모델 + 승격(promote) 시 ID 체계 자동 부여** — Fleeting → Zettel ← Literature 구조로 Luhmann 스타일 alphanumeric ID를 자동 생성(숫자/문자 교차 분기: 1 → 1a → 1a1, 형제는 1a → 1b).
- **모든 노트 변경을 old/new 값과 함께 기록하는 감사 이력 시스템** — 생성·수정·링크·삭제 등 모든 mutation을 history 테이블에 기록하고 CLI로 조회, 끊어진 링크를 탐지하는 dangling 명령과 함께 지식 그래프의 변경 추적성을 확보. 도메인/인프라를 분리한 repository 구조로 주입형 DB 기반 테스트 가능성 확보.
- **Single-binary 배포에서 Web UI 정적 자산 위치를 다단 fallback으로 탐색** — Homebrew, curl 설치, 개발 모드 모두 동일 바이너리로 동작하도록 환경 변수 → 사용자 디렉토리 → Homebrew prefix → 실행 파일 인접 경로 → 소스 경로 순으로 resolver를 직접 구현.

□ **Stack** · Bun · TypeScript · SQLite (FTS5) · React + Cytoscape.js (Web UI)

SKILLS

Languages	TypeScript, JavaScript, Java Go, Rust (basics)
Backend	NestJS, Fastify, Elysia (Bun), Spring Boot
Database	PostgreSQL, MySQL, MariaDB, Redis, SQLite (FTS5)
Web3	viem, ethers, EVM (Abstract / Aptos / BNB / Klaytn), EIP-1271/712, NFT indexing
Infra & DevOps	AWS (ECS/EKS/RDS/CloudFront), GCP (Cloud Run), Kubernetes, Helm, Docker, Terraform, GitHub Actions (OIDC), ArgoCD
Observability	Loki, Prometheus, Grafana, Langfuse, Redash

AWARDS

1st	Klaymakers22 Hackathon	KUP
2nd	BNB Hackathon Track #1: The Road towards Decentralized Society	R3plica
1st	Coinbase Build The Future Track — Open Ideas	R3plica
1st	Aptos Seoul Hack 2023 — Gaming Track	AptoPlay (aptoPlay-core lib 제작)

EDUCATION

서울과학기술대학교 · 컴퓨터공학 학사	2014.02 — 2021.02
----------------------	-------------------

CERTIFICATE · LANGUAGES

리눅스 마스터 2급	2015
------------	------

